Identifying LLMs from Their Responses

Team leader: Sanyam Agrawal

Team member: Shaan Patel

Problem and dataset

The goal was to see whether a CNN and an RNN could identify responses from GPT, Claude, or Gemini. We also tested whether including the prompt helped, whether the classifiers worked on new types of questions, and which response characteristics helped distinguish the sources.

We used 36 responses, with 12 from each family. Each model answered the same 12 prompts: four explanations, four everyday advice questions, and four math or logic questions. The team reported the model labels as GPT 6.1 Sol Medium, Claude Sonnet 5.5 Medium, and Gemini 3.6 Flash. These exact versions were not independently verified.

The responses came from the team's three response PDFs. We removed interface text, checked screenshot text after OCR, and kept the readable part of one clipped Gemini answer. No missing text or additional responses were invented. All four questions use the same reconstructed dataset. The team used one ongoing chat per model; other generation settings are unknown.

Models and training

We used Python and PyTorch to build a CNN and an LSTM. The LSTM is the RNN used in this project. Both classifiers predicted one of three labels: GPT, Claude, or Gemini.

The CNN represented each token with 32 numbers. It had one convolution layer with 32 filters and a window of three tokens. It used ReLU and max pooling, followed by dropout and an output layer. Padding positions were excluded from pooling.

The LSTM used the same token representation and one LSTM layer with 32 hidden units. It used the final hidden state of the real token sequence, followed by dropout and an output layer. Packed sequences prevented padding from affecting that state.

Both classifiers trained from scratch for 30 epochs using Adam, a learning rate of 0.003, and a batch size of 9. Dropout was 0.2. We used cross entropy loss and gradient clipping at 1.0. Settings were fixed, with no validation set or selection based on test results.

We lowercased the text and split it into tokens containing letters and numbers. Punctuation was removed. The vocabulary came from training data only, with unseen words mapped to an unknown token. A separator divided the prompt and response when both were used. The length limit was 768 tokens, which was enough for every full record.

Testing

For Q1 and Q2, we compared prompt only, response only, and prompt plus response. We used four testing rounds, with 27 training responses and 9 test responses per round. Each round held out three questions, one per category. All answers to the same question stayed together, so no test question appeared in that round's training data.

Each response was tested once in each setup. We measured accuracy and macro F1. Macro F1 gives equal weight to each family. Random guessing would average 33.3% accuracy because the three labels have equal numbers of responses.

Results

Classifier	Input used	Accuracy	Macro F1
CNN	Prompt only	33.3%	0.272
CNN	Response only	47.2%	0.440
CNN	Prompt + response	50.0%	0.463
LSTM	Prompt only	33.3%	0.233
LSTM	Response only	44.4%	0.416
LSTM	Prompt + response	44.4%	0.433

RQ1 Identifying the model from its response

Using responses alone, the CNN got 17 of 36 correct, and the LSTM got 16 of 36 correct. Both scored above the 33.3% guessing average but still made many mistakes. The CNN did slightly better, with just one more correct prediction. These results do not show that either classifier can reliably identify the source of a response.

RQ2 Does the prompt help

Using only the prompt gave 33.3% accuracy for both classifiers. Each identical prompt appeared with all three labels, so the classifier made the same prediction for those three inputs and got one right. The prompt alone could not distinguish the source.

Adding the prompt improved CNN accuracy from 47.2% to 50.0%, which was one more correct answer. LSTM accuracy stayed at 44.4%, although macro F1 increased from 0.416 to 0.433. The improvement was too small to conclude that including the prompt consistently helps.

If prompt only accuracy were unusually high in another dataset, different models might have received different topics or templates. A classifier could learn those collection differences instead of how each model writes.

RQ3 Do the classifiers work on a new type of question

We trained the CNN and LSTM on two categories and tested on the third, rotating explanations, everyday advice, and math or logic. Each run used 24 training responses and 12 test responses. We used response text only, built the vocabulary from training data, and kept the earlier model and training settings.

Classifier	Test category	Accuracy	Macro F1
CNN	Explanation	33.3%	0.269
CNN	Everyday advice	33.3%	0.254
CNN	Math or logic	33.3%	0.267

LSTM	Explanation	33.3%	0.276
LSTM	Everyday advice	33.3%	0.167
LSTM	Math or logic	33.3%	0.281

Both classifiers got 4 of 12 answers correct in every category, matching the guessing average. Compared with Q1, CNN accuracy fell by 13.9 percentage points and LSTM accuracy fell by 11.1 points. They did not reliably identify the source on a new type of question.

No family was consistently recognizable across tasks. For example, the LSTM predicted Gemini for every advice answer but missed every Gemini answer on math or logic questions. Recognizing one family well in one category did not mean it worked across categories.

The results did not establish writing patterns that worked across tasks. Each category had only four questions, and training used 24 responses instead of 27. The drop could reflect both the new task and the smaller training set.

RQ4 What characteristics distinguish the responses

We compared answer length and lexical diversity using the same tokenizer as the classifiers. Lexical diversity is the number of different tokens divided by the total tokens in an answer. The table shows the average for each family.

Model family	Average tokens	Average lexical diversity
GPT	81.0	0.718
Claude	264.7	0.568
Gemini	301.2	0.518

GPT gave shorter answers and had the highest lexical diversity. However, longer answers have more chances to repeat words, so this does not establish a wider vocabulary. Length is a structural difference, while lexical diversity measures vocabulary variety.

A classifier using only answer length got 19 of 36 answers correct, giving 52.8% accuracy and 0.487 macro F1. It learned each family's average log token count from training data and chose the closest average for each test answer. It used the same four question groups for testing, without tuning thresholds on test results. Length was useful in this collection, but this does not prove that the neural models relied on it.

We also retrained CNN and LSTM using only the first 20 tokens of each answer. All answers had at least 20 tokens, so the inputs were equally long. Training settings, test groups, and random seeds stayed the same.

Classifier	Full answer accuracy	First 20 tokens accuracy
CNN	47.2%	30.6%
LSTM	44.4%	30.6%

Accuracy fell by 16.7 percentage points for CNN and 13.9 points for LSTM. Shortening the answers removed content as well as length differences, so the drop cannot be attributed to length alone. Punctuation and formatting were not tested because the tokenizer removes them. These results only apply to the responses we tested.

Lessons learned

Both classifiers reached 100% training accuracy when using responses alone or prompt plus response, and in the Q3 experiments. Their much lower test accuracy showed overfitting: learning the training examples did not mean they could handle new questions or tasks.

The dataset had only 12 distinct questions. Shared chat histories, PDF extraction, and the clipped Gemini answer could also affect the results. We learned to keep test questions separate from training questions, compare training and test performance, and use simple methods to help interpret neural results. No statistical significance claim is made.